HackPad Electronics Guide

Schematic → PCB → manufacturing files, in KiCad

Device: 12 MX keys (4 × 3), 1 EC11 encoder with push, 0.91" OLED, 12 SK6812 MINI-E, Seeed XIAO RP2040
Pin plan: Documentation/feasibility.md (PR #1), reproduced in §2
You do the work. This guide explains what to do, why, and how to check it.

> **Nothing in this guide has been built or tested.**
> Every value is tagged: Certain checked against a datasheet or official doc, Likely strong engineering judgement, Guessing confirm before relying on it. When the guide and a datasheet disagree, the datasheet wins.

Contents

1. What you're building
2. The pin plan
3. How a key matrix works
4. The duplex matrix (your 13 switches on 6 pins)
5. KiCad setup and libraries
6. Drawing the schematic
7. ERC and footprint assignment
8. PCB layout
9. Routing
10. DRC and manufacturing files
11. Soldering order and first power-on
12. Checklists and common mistakes

TechPad project · MoonRock · 2026-10-16 · v1

1. What you're building

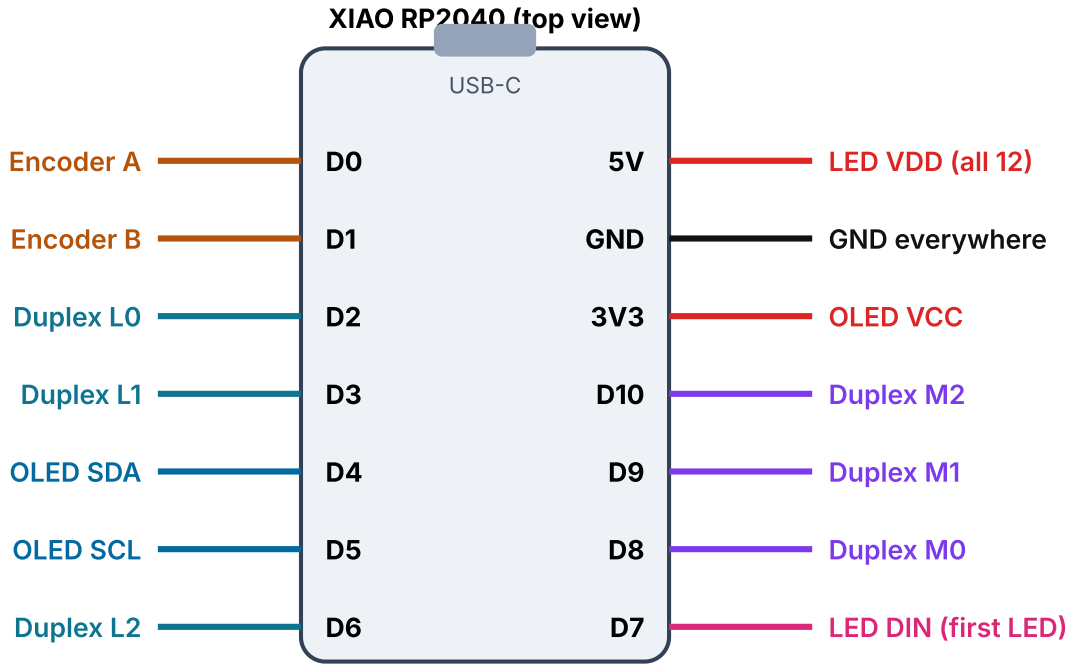
The PCB carries every electrical part. The case (later) is designed around it, so the board is the reference for all dimensions.

Part	Qty	Where it goes	Connects to
Seeed XIAO RP2040	1	Back of PCB, USB-C at the top edge	Everything
MX switch	12	Front, 19.05 mm grid, 4 × 3	Duplex matrix
1N4148 diode (through-hole)	13	Next to each switch + the encoder switch	Duplex matrix
EC11 encoder with push	1	Front, top-right	A/B → D0/D1, C → GND, push → matrix
0.91" OLED module (4-pin)	1	Front, top-left, on its header	3V3, GND, D5 (SCL), D4 (SDA)
SK6812 MINI-E	12	Back, reverse-mount, shining up through each switch	5V, GND, data chain from D7
M3 mounting holes	4	Near the corners	(mechanical)

> **Why a custom PCB and not hand-wiring?**
> 13 diodes, 12 LEDs and a 6-line matrix are about 80 solder joints. A PCB makes them repeatable, and Hackpad requires one (2-layer, ≤ 100 × 100 mm) Certain.

2. The pin plan

This is the official table from `Documentation/feasibility.md`. The XIAO RP2040 has exactly 11 usable GPIO (D0–D10), and this plan uses all of them Certain.



Physical pin order as on the Seeed wiki pinout (USB-C at the top).

XIAO pin	RP2040 GPIO	Function	Notes
D0	GPIO26	Encoder A	Internal pull-up in firmware
D1	GPIO27	Encoder B	Internal pull-up in firmware
D2	GPIO28	Duplex line L0	
D3	GPIO29	Duplex line L1	
D4	GPIO6	I²C SDA → OLED	I²C1 on the RP2040
D5	GPIO7	I²C SCL → OLED	
D6	GPIO0	Duplex line L2	
D7	GPIO1	SK6812 data in	To DIN of LED 1
D8	GPIO2	Duplex line M0	
D9	GPIO4	Duplex line M1	
D10	GPIO3	Duplex line M2	

The XIAO is not 5 V tolerant.
Nothing that can carry 5 V may touch a D-pin. That's why the OLED runs from 3V3, not 5V Certain (Seeed wiki).

Pins can still be swapped while you route.
If two traces cross awkwardly, swapping, say, L1 and L2 is fine, because the firmware simply follows whatever you wire. Update the table in feasibility.md when you do, so the firmware matches.

3. How a key matrix works

The problem

The simplest wiring gives each switch its own pin: one side to the pin, the other to GND. 13 switches would need 13 pins, and you only have 6 left after the encoder, OLED and LEDs.

The idea: rows and columns

Put the switches on a grid. Every switch connects one **row** wire to one **column** wire. With 3 rows and 3 columns (6 pins) you can tell 9 switches apart, because each switch sits on a unique row–column pair.

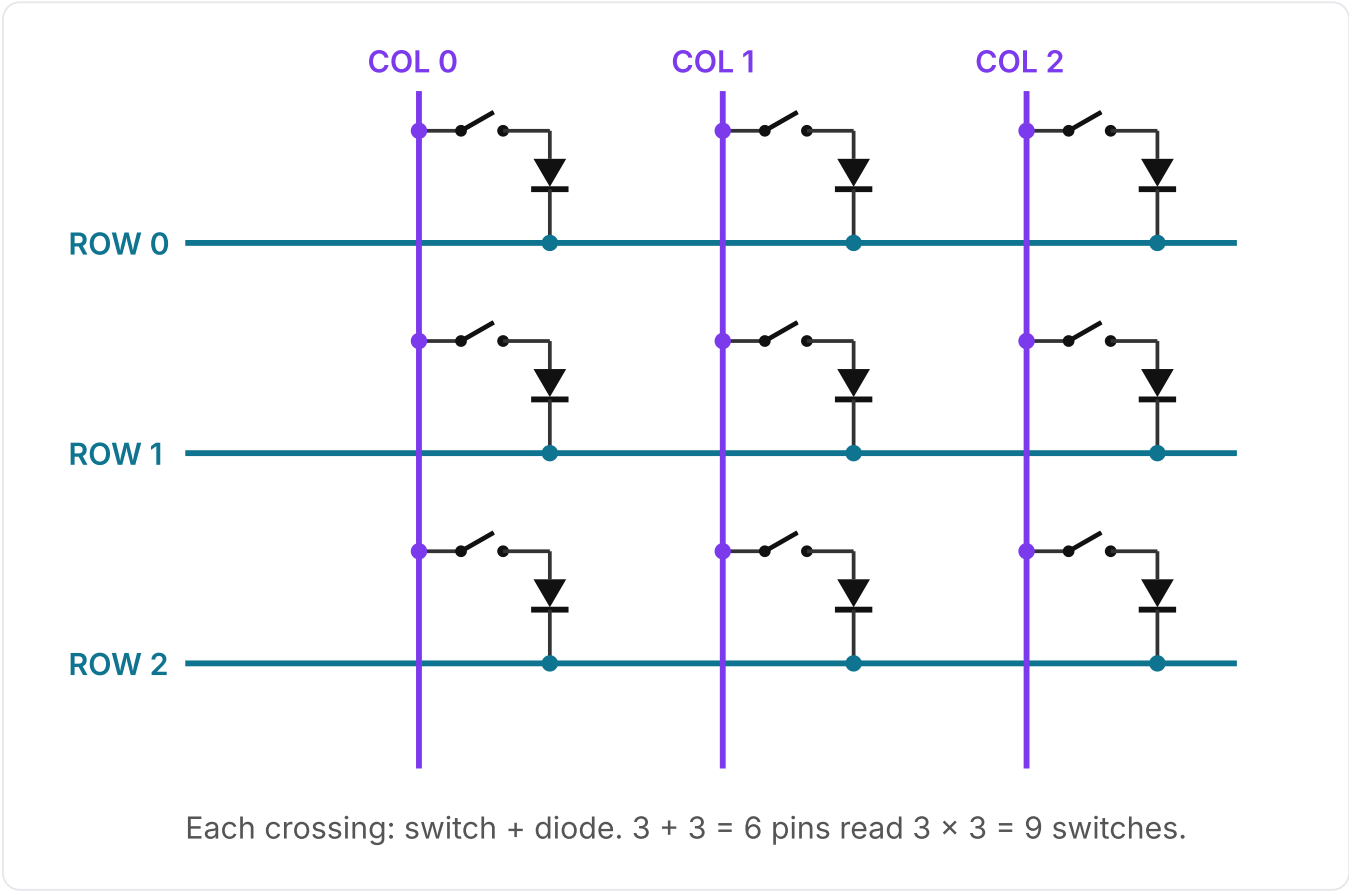


Figure 1. A normal 3 × 3 matrix. The diode's bar (cathode) points toward the row.

How the firmware reads it (scanning)

1. All column pins are **inputs with pull-ups**, so they read HIGH when nothing is pressed.
2. The firmware drives **one row LOW**, and leaves the others floating (high-impedance).
3. It reads the columns. A column that reads LOW has a pressed switch on *that* row.
4. It moves to the next row. One full scan takes well under a millisecond, so it repeats about 1000 times a second.

Current path when a switch is pressed: pull-up on the column → switch → diode → the LOW row. The diode only lets current go that way, so its cathode (the bar) faces the row. In QMK this is called `COL2ROW`.

Why the diodes? (ghosting)

Without diodes, pressing three switches at three corners of a rectangle creates a path backwards through the third switch. The firmware sees a fourth key at the fourth corner that you never pressed. That phantom is a

"ghost". A diode on every switch blocks the backwards path, so every combination of keys reads correctly Certain.

> **Rule to remember**
> Current flows from the input (pulled HIGH) to the output being driven LOW. The diode's bar always points at the line that gets driven LOW when that switch is supposed to be read.

---

4. The duplex matrix (your 13 switches on 6 pins)

A normal matrix on 6 pins reads 9 switches. You have 13: 12 keys plus the encoder's push. The duplex matrix doubles the capacity without extra pins or chips.

The trick

At each crossing, put **two** switches, each with its own diode, and point the two diodes **in opposite directions**. Call the two groups of lines **L** (L0–L2) and **M** (M0–M2). Neither is "rows" or "columns" any more, because each group takes a turn at both jobs.

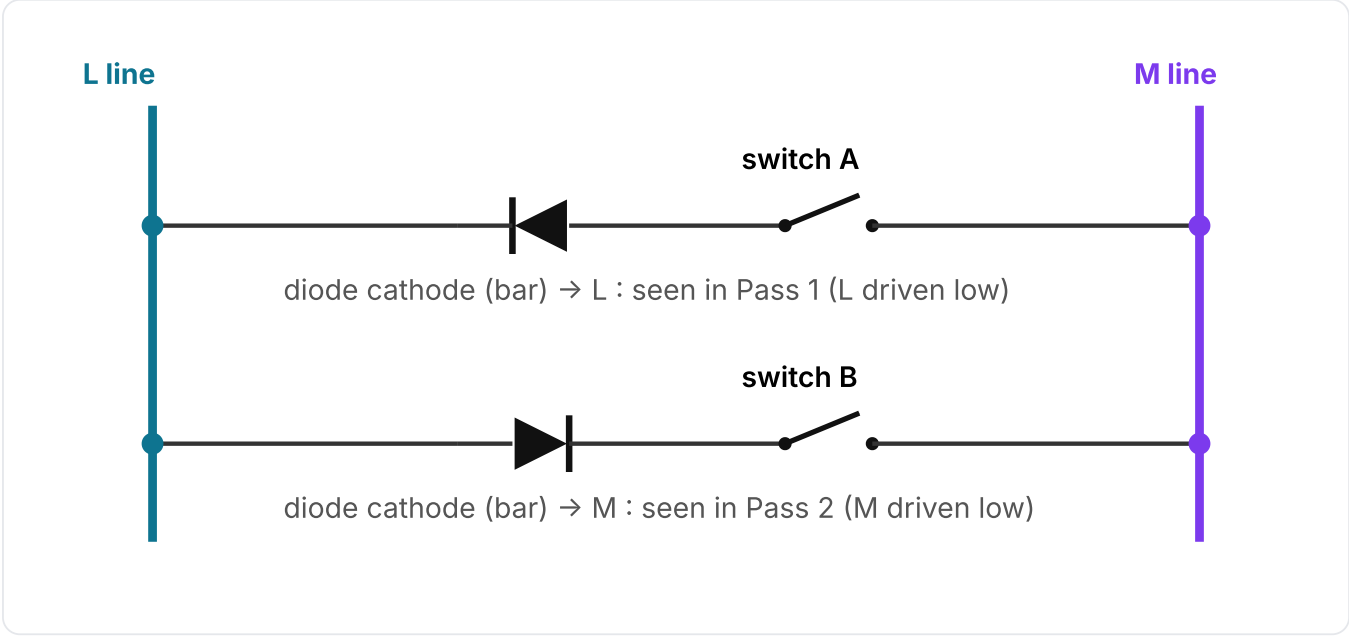


Figure 2. One crossing of the duplex matrix. Two switches share the same two wires, but their diodes face opposite ways.

The firmware scans in two passes:

Pass	Driven LOW (one at a time)	Read (pull-ups on)	Sees switches whose diode bar points to…
1	L0, L1, L2	M0, M1, M2	L
2	M0, M1, M2	L0, L1, L2	M

In Pass 1, switch A conducts (current flows from the M pull-up through switch A and its diode into the LOW L line). Switch B's diode blocks, so it's invisible. In Pass 2 it's the reverse. So each crossing gives you 2 switches: 3 × 3 × 2 = **18 positions** on 6 pins Certain.

Your switch map (official)

Crossing	Pass 1 — diode bar → L	Pass 2 — diode bar → M
L0 × M0	K(0,0)	K(0,3)
L0 × M1	K(0,1)	**Encoder push**
L0 × M2	K(0,2)	spare
L1 × M0	K(1,0)	K(1,3)
L1 × M1	K(1,1)	spare
L1 × M2	K(1,2)	spare
L2 × M0	K(2,0)	K(2,3)
L2 × M1	K(2,1)	spare
L2 × M2	K(2,2)	spare

K(row, col): row 0 is the top row, col 0 the left column. Notice the pattern: columns 0–2 are a normal 3 × 3 matrix (Pass 1), and column 3 plus the encoder push ride along in Pass 2.

Two things that break a duplex matrix

1. **A diode in backwards.** The switch moves to the other pass, so the wrong key fires (or a "spare" fires). Fix: put the cathode-bar symbol on the silkscreen next to every diode and check each one before soldering.
2. **Lines not floating.** When a line isn't being driven, the firmware must set it as an input, never as a HIGH output. Otherwise the passes interfere. This is firmware's job, not the PCB's.

Firmware note (for later)

QMK has no built-in duplex scan. You'll write a short `matrix_scan_custom()` (about 50 lines) using QMK's "lite" custom matrix; QMK still handles debounce and keycodes Certain (docs.qmk.fm/custom_matrix). The PCB doesn't change because of this.

5. KiCad setup and libraries

1. **Install KiCad 8 or 9.** Either works; the steps below use menu names from both.
2. **Create the project inside your repo:** `My_TechPad/PCB/HackPad.kicad_pro`. Hackpad expects `PCB/` to contain the `.kicad_pro`, `.kicad_sch` and `.kicad_pcb` files Certain.
3. **Install the Hack Club "KiCad care package"** (already in your `assets\kicad_care_package` folder). Follow §5.1.
4. **Everything else comes from KiCad's built-in libraries.** Full list:

Part	Symbol	Footprint (KiCad built-in unless noted)
XIAO RP2040	`hackpad_xiao:MOUDLE-SEEEDUINO-XIAO`	`hackpad:XIAO-Generic-Hybrid-14P-2.54-21X17.8MM`, flipped to the back
MX switch	`Switch:SW_Push`	`Button_Switch_Keyboard:SW_Cherry_MX_1.00u_PCB`
1N4148	`Diode:1N4148`	`Diode_THT:D_DO-35_SOD27_P7.62mm_Horizontal`
EC11 encoder	`Device:RotaryEncoder_Switch`	`Rotary_Encoder:RotaryEncoder_Alps_EC11E-Switch_Vertical_H20mm`
OLED	`Connector_Generic:Conn_01x04`	`Connector_PinHeader_2.54mm:PinHeader_1x04_P2.54mm_Vertical`
SK6812 MINI-E	`hackpad_led:SK6812MINI-E`	`hackpad:SK6812MINI-E_fixed` (has the cut-out)
Mounting hole	`Mechanical:MountingHole`	`MountingHole:MountingHole_3.2mm_M3`

5.1 Installing the symbol and footprint libraries

KiCad keeps two separate kinds of library. A **symbol library** is one file ending in `.kicad_sym`; it holds schematic drawings. A **footprint library** is a *folder* ending in `.pretty`; each `.kicad_mod` file inside is one footprint. They are installed in two different places.

Your care package (`B:\Aarav\My_TechPad\assets\kicad_care_package`) contains exactly Certain:

File	Kind	What's inside
`SK6812MINI-E.kicad_sym`	Symbol library	Symbol **SK6812MINI-E**: pin 1 VDD, 2 DOUT, 3 VSS (GND), 4 DIN
`XIAO_RP2040.kicad_sym`	Symbol library	Symbol **MOUDLE-SEEEDUINO-XIAO** (the typo is theirs): pins 1–11 = D0–D10, 12 = 3V3, 13 = GND, 14 = 5V
`footprints.pretty/`	Footprint library	`SK6812MINI-E_fixed` (reverse-mount, includes the board cut-out on Edge.Cuts) and `XIAO-Generic-Hybrid-14P-2.54-21X17.8MM` (14 through-hole pads)

a. **Create the KiCad project in `B:\Aarav\My_TechPad\PCB\`** (File → New Project → name it `HackPad`). The care package can stay in `assets/`; you'll point to it with a relative path.

b. **Symbols.** With the project open: *Preferences* → *Manage Symbol Libraries* → *Project Specific Libraries* tab → folder icon (*Add existing library*) → select `SK6812MINI-E.kicad_sym`. Repeat for `XIAO_RP2040.kicad_sym`. Set the nicknames to `hackpad_led` and `hackpad_xiao`.

c. **Footprints.** *Preferences → Manage Footprint Libraries → Project Specific Libraries* tab → folder icon → select the **`footprints.pretty` folder itself** (not a file inside it). Change the nickname from `footprints` to `hackpad`.

d. **Make the paths portable.** In each table, edit the *Library Path* so it starts with `${KIPRJMOD}`:
`${KIPRJMOD}/../assets/kicad_care_package/SK6812MINI-E.kicad_sym`
`${KIPRJMOD}/../assets/kicad_care_package/XIAO_RP2040.kicad_sym`
`${KIPRJMOD}/../assets/kicad_care_package/footprints.pretty`
`${KIPRJMOD}` means "the folder containing HackPad.kicad_pro" (here `PCB\`), and `..` goes up one level to `My_TechPad\`. A path like `B:\Aarav\…` only works on your PC Certain.

e. **Test.** In the schematic press `A` and search `SK6812MINI-E` and `XIAO`. In *Tools → Assign Footprints*, a library called `hackpad` should list both footprints.

f. **Commit** `assets/kicad_care_package/` plus `PCB/sym-lib-table` and `PCB/fp-lib-table` (KiCad creates these when you add project libraries). Without them, anyone who clones the repo sees "missing library" errors.

Two things the care-package symbols don't do for you

1. **Neither symbol has a working footprint linked.** The XIAO's Footprint field is empty, and the LED's says `SK6812MINI-E - 3.2x2.8x1.78mm`, which isn't a real library link. Assign them by hand: `hackpad:XIAO-Generic-Hybrid-14P-2.54-21X17.8MM` and `hackpad:SK6812MINI-E_fixed` Certain.
2. **The XIAO symbol's pin names are from a different XIAO** (e.g. `PA02_A0_D0`, `PB09_A7_D7_RX` are SAMD21 names). Only the **D-number** matters. Wire by D-number using the table in §2 and ignore the "PA…/PB…" part. The real RP2040 GPIO numbers are in §2.

	Project-specific (use this)	Global
Where it's saved	`PCB/sym-lib-table`, `PCB/fp-lib-table`	Your KiCad settings on this PC only
Travels with the repo	Yes	No
Good for	Hackpad parts, anything a reviewer needs	Libraries you reuse in every project

Library gotchas

- If a library shows a red/yellow warning icon in the table, its path is wrong: fix the path, don't delete the row.
- If KiCad offers to convert an old `.lib` file, accept; it saves a `.kicad_sym` next to it.
- If the care package includes 3D models (`.step` / `.wrl`), keep them in `lib/` too, so the 3D viewer and your board STEP export show real parts.
- Blueprint's warning: the LED part is **SK6812MINI-E**, not SK6812MINI. The symbol *and* footprint must both say MINI-E Certain.

Check the SK6812 MINI-E footprint carefully.

KiCad's built-in SK6812 MINI footprint is for the normal top-mount version, not the reverse-mount "-E". Use `SK6812MINI-E_fixed` from the care package, which has the cut-out. Its pads are 1 VDD, 2 DOUT, 3 VSS, 4 DIN, matching the symbol Certain; still compare the corner marking with the LED datasheet before soldering.

6. Drawing the schematic

Open the schematic editor. Work block by block; after each block, save and run ERC (§7). Use **net labels** (shortcut `L`) for L0–L2, M0–M2, SDA, SCL, LED_DIN, ENC_A, ENC_B instead of long wires. It keeps the sheet readable and is how reviewers expect a matrix to look.

6.1 The XIAO

1. Place the XIAO symbol. Attach a net label to each D-pin following §2.
2. Put a `+5V` power symbol on its 5V pin, `+3V3` on 3V3, `GND` on GND.
3. Add **PWR_FLAG** symbols on +5V and GND (and +3V3). The XIAO is powered over USB, which KiCad can't see, so without these ERC complains "power input not driven" Certain.

6.2 The duplex matrix (13 switches + 13 diodes)

Draw it as 9 small "crossing" blocks, one per L × M pair. Each block has one or two switch + diode branches.

1. **Pass 1 branch** (all 9 crossings): `M-label → switch → diode anode … diode cathode → L-label`. The diode bar points at the L line.
2. **Pass 2 branch** (only L0×M0, L0×M1, L1×M0, L2×M0): `L-label → switch → diode anode … diode cathode → M-label`. The bar points at the M line.
3. Annotate the switches so references match the key names: SW1…SW12 for K(0,0)…K(2,3) left to right, top to bottom, and the encoder push stays inside the encoder symbol. Diodes D1…D13 next to their switch.
4. For the encoder push: the encoder symbol's two switch pins (often S1/S2) are one "switch". Wire `L0 → S1`, `S2 → diode anode`, `diode cathode → M1`.

> **Self-check**
> Count: 13 diodes. 9 of them have their cathode on an L net, 4 on an M net. If your counts differ, a branch is backwards.

6.3 The encoder (turning)

Encoder A → `ENC_A` (D0), B → `ENC_B` (D1), common C (middle pin of the 3-pin side) → GND. No resistors needed; the RP2040's internal pull-ups are used Likely.

6.4 The OLED

Conn_01×04 with pins in the module's physical order: **1 GND, 2 VCC → +3V3, 3 SCL → D5, 4 SDA → D4** Certain (from your kit sheet). Getting this order wrong is the most common Hackpad mistake. Before ordering, hold the real module over the printed footprint and compare.

No pull-up resistors: these modules almost always have them on board Likely. Check yours for two small resistors near the pins (often marked 472 = 4.7 kΩ).

6.5 The LED chain

1. Place 12 SK6812 symbols. Each: VDD → +5V, GND → GND.
2. Chain the data: `D7 → DIN(LED1)`, `DOUT(LED1) → DIN(LED2)` … `DOUT(LED11) → DIN(LED12)`. LED12's DOUT stays unconnected (mark it with a no-connect flag, shortcut `Q`).
3. Order: put LED1 under K(0,0) and snake through the board (row 0 left→right, row 1 right→left, row 2 left→right). Snaking keeps data traces short. Write the order on the schematic, because the firmware's LED-to-key table (`g_led_config`) needs it for the "glow when pressed" effect.

> **Power and data level**
> • 12 LEDs at full white ≈ 720 mA, more than a USB port's 500 mA. Firmware caps brightness at ~40% (≈ 290 mA) Likely. Use 0.5 mm+ traces for 5V/GND.
> • The LEDs run on 5 V but D7 outputs 3.3 V. That's just under the LED's 0.7 × VDD = 3.5 V logic threshold. It usually works Likely. Optional safety net: power LED1 through a spare 1N4148 (5V → anode, cathode → LED1 VDD). The ~0.7 V drop lowers LED1's threshold to ~3.0 V, and LED1 then re-sends clean 5 V data to the rest.

6.6 Mounting holes

Add 4 `MountingHole` symbols (no connection). They only exist so the footprints appear on the PCB.

---

7. ERC and footprint assignment

1. **Annotate** (Tools → Annotate Schematic). Keep SW1–SW12 matching key order.
2. **Run ERC** (Inspect → Electrical Rules Checker). Target: 0 errors. Typical errors and fixes: "pin not connected" → wire it or add a no-connect flag; "power input not driven" → add PWR_FLAG; "label not connected" → typo in a net label (L0 vs LO is a classic).
3. **Assign footprints** (Tools → Assign Footprints) using the table in §5.
4. **Screenshot the schematic** for your README now (Hackpad requires it) Certain.

> **Stop and review here.**
> Feasibility.md says the PCB is laid out only after the schematic is reviewed. Post the schematic screenshot in the project thread; a second pair of eyes on diode directions and the OLED pin order is cheap now and expensive later.

8. PCB layout

Open the PCB editor and import the schematic (Tools → Update PCB from Schematic, F8). Everything lands in a pile; you'll place it by hand.

8.1 Board outline

On the **Edge.Cuts** layer, draw a 92 × 87 mm rectangle with 3 mm rounded corners. That's under the 100 × 100 mm limit and leaves room for 3 mm case walls on a ~100 × 95 mm case Likely. Set the drill/place origin at the top-left corner (Place → Drill/Place File Origin) so the coordinates below match.

8.2 Placement (concept coordinates, top-left origin)

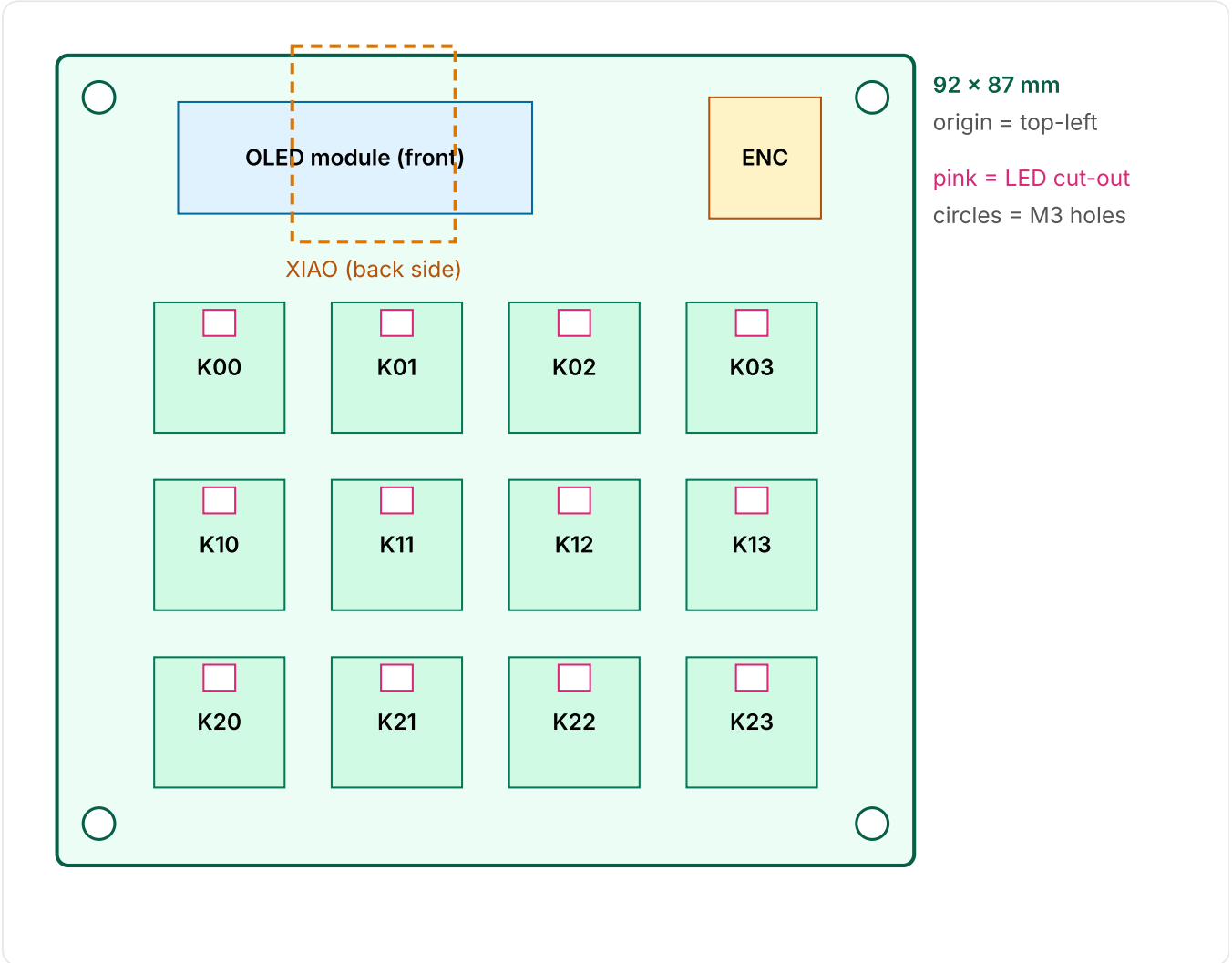


Figure 3. Suggested placement. Final positions are yours; keep the 19.05 mm key spacing exact.

Part	Position (centre, mm)	Side	Why
Key columns	x = 17.425, 36.475, 55.525, 74.575	Front	19.05 mm pitch, centred on 92 mm
Key rows	y = 33.525, 52.575, 71.625	Front	Leaves the top 25 mm for OLED + knob
Encoder	(76, 11)	Front	Top-right, over the key columns' right side
OLED header	at the left end of a 38 × 12 mm module centred near (32, 11)	Front	Top-left; measure your module first
XIAO	x ≈ 34, USB-C flush with the top edge	Back	Short cable path; port reaches the case's back wall
M3 holes	(4.5, 4.5), (87.5, 4.5), (4.5, 82.5), (87.5, 82.5)	—	The case is designed later to match these
Diodes	Beside each switch, all in the same orientation	Front or back	Same direction = easy visual check
LEDs	Under each switch's LED window	Back (reverse-mount)	Light passes up through the switch

Placing keys exactly
Set a custom grid of 19.05 mm (View → Grid Properties), or select a switch and press `M` then use "Move Exactly" / edit the position fields directly. Do not eyeball the spacing: keycaps will rub if the pitch is off by even 0.5 mm.

Things that collide
• **XIAO pins under the OLED.** The XIAO is on the back, but its through-hole pins poke out the front. If the OLED module sits over them, flush-cut the pins or keep the OLED on its header standoff Likely.
• **LED window side.** MX-style switches have the LED window on one side of the centre. Check which side on your kit switches and put the LED footprint there. If the Hack Club MX footprint already includes an LED position, use it Likely.
• **Encoder legs.** The EC11's two big mounting tabs need their slotted holes; don't delete them from the footprint.

9. Routing

Net type	Track width	Reason
Signals (L/M lines, I²C, encoder, LED data)	0.25 mm	Tiny currents; easy to fit between pads
+5V and GND to LEDs	0.5 mm or wider	Up to ~0.3 A at the brightness cap

1. Set these as **Net Classes** (File → Board Setup → Net Classes): Default 0.25 mm, "Power" 0.5 mm for +5V, GND, +3V3.
2. Route the **LED data chain** first, short and direct, LED to LED.
3. Route the **matrix**: L lines mostly horizontal on one layer, M lines mostly vertical on the other, with vias where they change layer. That's the classic way to avoid crossings.
4. Route encoder and OLED lines.
5. Add a **GND copper pour** on both layers (Place → Add Filled Zone, net GND, whole board). Then press `B` to fill. Stitch the two pours with a few vias. A ground pour gives every return current a short path and reduces

noise Likely.

6. Keep 5V as a thick track (not a pour) running along the LED rows.

> **Via size for JLCPCB**
> 0.3 mm drill / 0.6 mm diameter is a safe default. Set it in Board Setup → Pre-defined Sizes. JLCPCB's standard 2-layer limits are about 0.127 mm track and spacing Likely; staying at 0.25 mm keeps you far from them.

---

10. DRC and manufacturing files

1. **Run DRC** (Inspect → Design Rules Checker) with "Refill all zones" ticked. Target: 0 errors, and read every warning. Unconnected-item errors mean a net isn't fully routed.
2. **Look at it in 3D** (View → 3D Viewer, Alt+3). Check that the XIAO is on the back, the USB-C reaches the edge, and nothing overlaps.
3. **Silkscreen**: add "HackPad", your name, a version (v1.0), and the date. Make sure every diode bar is visible.
4. **Export Gerbers** (File → Fabrication Outputs → Gerbers): layers F.Cu, B.Cu, F.Paste, B.Paste, F.Silkscreen, B.Silkscreen, F.Mask, B.Mask, Edge.Cuts. Then **Generate Drill Files** (Excellon, same folder). Zip the folder as `production/gerbers.zip` Certain (Hackpad repo structure).
5. **Check the zip** by uploading it to JLCPCB's quote page (don't pay yet). Its viewer shows the board as they'll make it; look for missing outline or holes.
6. **Export a STEP model** of the board (File → Export → STEP) into `CAD/`. You'll import it into Fusion 360 to design the case around the real board.
7. **Screenshots** of the PCB (front, back, 3D) for the README Certain.

---

11. Soldering order and first power-on

Low parts first, tall parts last, so the board lies flat while you work.

1. **SK6812 MINI-E LEDs** (back, reverse-mount). Hardest part: low iron temperature (~300 °C), quick touches; they're heat-sensitive Likely. Check the corner marking matches the silkscreen.
2. **Diodes.** Bar matches the silkscreen bar. Check all 13 before trimming legs.
3. **XIAO** on the back. Solder two opposite corners, check it's flat, then the rest.
4. **OLED header**, then the OLED. Check pin order one more time.
5. **Encoder**, then **switches** (after the switch plate is in place, if your case design needs the plate first).

> **First power-on, step by step**
> 1. Before plugging in: multimeter in continuity mode, check 5V–GND and 3V3–GND are *not* shorted.
> 2. Plug in. The XIAO's LED should light; nothing should get warm.
> 3. Flash a minimal firmware that tests one thing at a time: one key, then the matrix, then the encoder, OLED, LEDs (Milestone 3 in your brief).

12. Checklists and common mistakes

Before you send the PCB to manufacture

- ☐ ERC: 0 errors. DRC: 0 errors, zones refilled.
- ☐ Board ≤ 100 × 100 mm, 2 layers.
- ☐ OLED footprint order GND-VCC-SCL-SDA, and VCC is on +3V3.
- ☐ 13 diodes: 9 cathodes on L nets, 4 on M nets, matching the switch map in §4.
- ☐ No D-pin connected to anything at 5 V.
- ☐ LED chain D7 → LED1 → … → LED12, order written down.
- ☐ Switch pitch exactly 19.05 mm in both directions.
- ☐ XIAO on the back with USB-C at the board edge.
- ☐ Encoder C pin on GND; A on D0, B on D1.
- ☐ Mounting holes match what you'll put in the case.
- ☐ Pin table in feasibility.md updated if you swapped any pins.
- ☐ Schematic, PCB and 3D screenshots saved for the README.

Common mistakes

Mistake	Symptom	Prevention
OLED pin order mirrored	Blank screen, module gets hot	Compare footprint with the real module before ordering
Diode backwards	Wrong key fires, or a "spare" fires	Silkscreen bars, same orientation everywhere
XIAO footprint mirrored (placed on back incorrectly)	Nothing works	Flip the footprint with KiCad's "Flip" (F), never redraw it; check in 3D
LED footprint for the wrong variant	LED doesn't fit or faces the wrong way	Use a MINI-E reverse-mount footprint with a cut-out
Net label typo (L0 vs LO)	Line split in two	ERC "label not connected" warnings
Thin 5V tracks	LEDs dim or flicker at the far end	Power net class 0.5 mm+

Where things go in the repo

Folder	Contents
`PCB/`	HackPad.kicad_pro, .kicad_sch, .kicad_pcb, lib/
`production/`	gerbers.zip (later: firmware .uf2, case STEP files)
`CAD/`	Board STEP for case design
`Documentation/`	feasibility.md (pin table), screenshots

Sources used

Seeed XIAO RP2040 wiki (pins, 3.3 V logic) · Blueprint Hackpad submission guide and parts list · Stardance Hackpad guide (case rules, KiCad flow) · QMK custom matrix docs · your kit list (OLED pin order) · Documentation/feasibility.md (pin and switch tables).